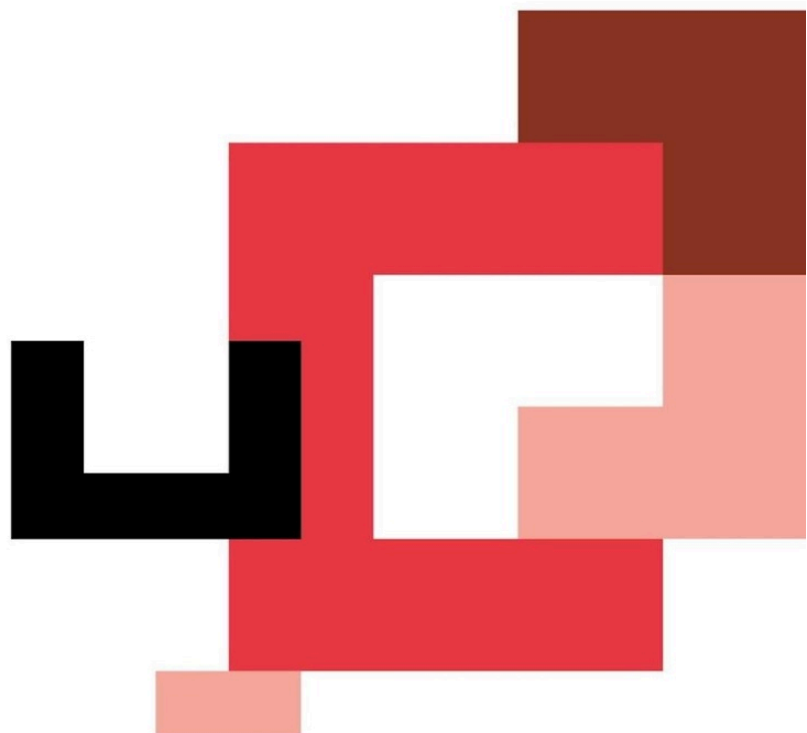




FULL STACK DEVELOPER + IA

Módulo 3 | Feature 3

Redux Toolkit, auth real y
estado global



Objetivo de la feature

Introducir un estado global en la aplicación y gestionar la autenticación real, entendiendo:

- Cómo funciona **Redux Toolkit**
 - Cómo centralizar el estado con un **store**
 - Cómo gestionar autenticación con **JWT**
 - Cómo trabajar con **acciones síncronas y asíncronas**
 - Cómo proteger rutas con **PrivateRoute**
 - Cómo gestionar estado global de **carrito** y **wishlist**
-

Contexto del sprint

En el Sprint 2 ya conectamos el frontend con la API.

En este sprint:

- Dejamos de manejar estado local disperso
- Centralizamos toda la lógica en Redux
- Implementamos autenticación real
- Añadimos funcionalidades completas de usuario

Esto convierte la app en una aplicación estructurada y escalable.

Aquí tienes el ejemplo que puedes tomar como guía. Navega por la página:

<https://shop-example-react.netlify.app/sprint-3>

Requisitos

Tech / configuración

- Backend corriendo en `http://localhost:3000`
- React + Vite
- Axios ya configurado
- Redux Toolkit

Shell

```
npm install @reduxjs/toolkit react-redux
```

Estructura sugerida

Shell

src/

api/

 axios.js

 auth.js

 cart.js

 wishlist.js

 products.js

 reviews.js

store/

 index.js

 authSlice.js

 cartSlice.js

 wishlistSlice.js

components/

 PrivateRoute/

 CartItem/

 CartSummary/

 WishlistButton/

 ReviewForm/

pages/

 CartPage/

 CheckoutSuccessPage/

```
WishlistPage/  
  
ProfilePage/  
  
hooks/  
  
  useReviews.js  
  
router/  
  
  index.jsx  
  
main.jsx
```

Qué hay que hacer (paso a paso)

1) Configurar Redux Store

- Crear `store/index.js`
- Usar `configureStore`
- Añadir slices

```
JavaScript  
configureStore({  
  
  reducer: {  
  
    auth: authReducer,  
  
    cart: cartReducer,  
  
    wishlist: wishlistReducer,  
  
  },  
  
})
```

- Envolver la app con `<Provider>` en `main.jsx`

Objetivo: tener estado global disponible.

2) Crear authSlice

- Estado:
 - token
 - user
 - loading
 - error
- Reducers:
 - logout
- Thunks:
 - login
 - register
- Guardar token en `localStorage`

Objetivo: gestionar autenticación global.

3) Crear cartSlice

- Estado:
 - items
 - loading
 - error
- Thunks:
 - fetchCart
 - addCartItem
 - removeCartItem
 - checkout

Objetivo: carrito sincronizado con backend.

4) Crear wishlistSlice

- Estado:
 - productIds
- Thunks:
 - fetchWishlist
 - toggleWishlist

Objetivo: favoritos persistentes.

5) Configurar Axios interceptors

- Inyectar token en cada petición
- Manejar errores 401

Objetivo: automatizar autenticación en requests.

6) Crear rutas privadas

Crear `PrivateRoute`:

- Si no hay token → redirect a `/login`
- Si hay token → renderiza contenido

Usar en router para:

- `/cart`
- `/wishlist`
- `/profile`
- `/checkout`

Objetivo: proteger contenido.

7) Conectar login y registro con Redux

- `LoginPage` → `dispatch(loginThunk)`
- `RegisterPage` → `dispatch(registerThunk)`

Objetivo: flujo real de autenticación.

8) Implementar carrito

- Añadir producto al carrito
- Eliminar producto
- Mostrar total

Objetivo: funcionalidad clave del e-commerce.

9) Implementar wishlist

- Botón toggle en productos

- Guardar en backend

Objetivo: interacción de usuario.

10) Añadir reseñas POST

- Crear ReviewForm
- Enviar reseñas autenticadas

Objetivo: completar flujo de producto.

IMPORTANTE: Esto solo si el tiempo nos lo permite.

Uso de la IA en este sprint

Para guiarte

- Explicación de Redux Toolkit
 - Revisión de slices
 - Debug de thunks
 - Flujo de autenticación
-

Prompts útiles

Redux Toolkit

Shell

Explícame cómo estructurar un slice en Redux Toolkit con `createSlice` y `createAsyncThunk`.

Auth

Shell

Estoy gestionando login con JWT en React + Redux.

¿Cómo debo guardar y usar el token correctamente?

Debug

Shell

Tengo un problema con `createAsyncThunk` que no actualiza el estado.

Te paso el código. Dime posibles causas.

Pistas

`createSlice`

Agrupar estado + reducers + acciones.

`createAsyncThunk`

Genera automáticamente:

- `pending`
- `fulfilled`
- `rejected`

Flujo típico

- `dispatch thunk`
- llamada API
- actualizar estado

`useSelector` / `useDispatch`

JavaScript

```
const data = useSelector(...)
```

```
dispatch(action())
```

Persistencia

Usar `localStorage` para mantener sesión.

Importante que tienen que tener en cuenta

No mezclar responsabilidades

- API → llamadas HTTP
 - Slice → estado
 - Component → UI
-

No duplicar estado

Si algo está en Redux → no lo repitas en useState.

Manejar loading y error

Siempre contemplar los tres estados.

Seguridad básica

El token debe enviarse en cada request.

Pensar en flujo

Login → token → store → UI → requests

Checks rápidos (autoevaluación)

- Login funciona y guarda token
- Al recargar, sigue logueado
- Rutas privadas funcionan
- Carrito añade y elimina productos
- Wishlist funciona
- Reviews se pueden enviar
- Si el token expira → redirige a login

Qué se prepara para el siguiente sprint

- Optimización de renders
- Cálculos derivados (useMemo)
- Mejora del flujo de compra
- Refinamiento de UI y experiencia

Enfoque del sprint

- Entender estado global
- Centralizar lógica
- Construir una app real

Este sprint marca el paso de aplicación simple a arquitectura profesional.